

Information-Driven MCMC Sampling for Path Planning

Sam Kelly

December 2025

Abstract

We present an MCMC-guided RRT framework that integrates Metropolis-Hastings sampling with belief-space planning and receding horizon navigation to address this gap. Our approach replaces uniform sampling with goal-biased samples drawn from a Boltzmann distribution over a cost function that balances goal distance, obstacle clearance, and information gain, while maintaining a Gaussian about the goal location. Experimental evaluation across 180 trials in maze environments of varying complexity reveals a nuanced performance profile. MCMC sampling produces significantly shorter paths when planning succeeds but outright fails more often than standard RRT for architectural reasons.

1 Introduction

The evolution of robotic motion planning techniques from deterministic searches to probabilistic, sampling-based methods represents one of the most distinct paradigm shifts in the field of autonomous systems. These planners play a key role in autonomous systems navigation pipelines, especially in global planning. At the beginning of this pipeline, autonomous systems use a variety of sensors to process raw data into a usable map, such as an occupancy grid, point cloud, or semantic map. After this occurs, agents must determine their position and orientation within the map using localization techniques, and global-level planning can proceed. Sampling-based planners do this by producing a sequence of collision-free waypoints. Finally, lower-level planners, such as Model Predictive Control (MPC), determine how to follow this path via a sequence of control actions, finding specific velocity, torque, and acceleration values that adhere to user constraints.

Sampling-based planning techniques have been applied to an array of problems in robotic motion planning, manufacturing, and even computational biology. The fundamental connection among all these applications is the goal of finding a continuous path in a high-dimensional configuration space that avoids regions defined by domain-specific constraints. To crystallize this idea, a seemingly unrelated application can be found in computational biology, where the

‘motion’ of a protein is modeled as a sequence of folding states, and the planner must find a path through a high-dimensional space of bond configuration that avoids undesirable outcomes such as high-energy states or overlapping atoms.

Despite the versatility of traditional sampling-based planners like RRT (Rapidly-Exploring Random Trees), their performance is inextricably linked to their underlying sampling regime. Basic variants of RRT rely on uniform sampling, which has the desirable property of being probabilistically complete. These variants leverage “Voronoi bias” to efficiently explore the configuration space, meaning that if a path exists, they are guaranteed to find one. However, treating the entire configuration space as equally important becomes exponentially more computationally expensive as dimensionality increases, where the region of optimal or even feasible solutions is a fraction of the entire space.

To address these issues, recent advancements in sampling-based planners have pivoted towards informed sampling techniques. This paper explores a novel synthesis of statistical mechanics and probabilistic motion sampling by replacing uniform sampling with a Metropolis-within-Gibbs sampler in order to sample from a predefined cost function. This is done by transforming our cost function into a probability density function via a Boltzmann distribution to sample from high-density regions. We then compare the two approaches across an array of efficiency, safety, and computational metrics in a series of clustered maze environments. The rest of this paper will discuss the specifics of our problem formulation, an overview of the methods used for our implementation, and a discussion of our results.

2 Problem Formulation

2.1 Environmental Geometry and Workspace

We consider the Rapidly-exploring Random Tree (RRT) in a 2D configuration space $\mathcal{X} \subseteq \mathbb{R}^2$, where the free space is defined as $\mathcal{X}_{\text{free}} = \mathcal{X} \setminus \mathcal{X}_{\text{obs}}$. The workspace is bounded such that $\mathcal{X} = [x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}] \subset \mathbb{R}^2$. Obstacle regions are characterized by $\mathcal{X}_{\text{obs}}$.

The objective is to incrementally grow a tree $\mathcal{T} = (V, E)$ from some initial condition $q_{\text{start}} \in \mathcal{X}_{\text{free}}$ toward a goal $q_{\text{goal}} \in \mathcal{X}_{\text{free}}$. The standard RRT algorithm samples uniformly from $\mathcal{X}_{\text{free}}$ and expands the tree toward sampled nodes via a steering function with step size η . It then collision-free edges until the goal is reached within some threshold ϵ .

For the purposes of this paper we focus on maze environments. Mazes pose significant challenges for path planning, requiring navigation through tight spaces and U-shaped, nested obstacles. We assume that $\mathcal{X}_{\text{free}}$ is connected, which guarantees that at least one collision-free path exists between q_{start} and q_{goal} . To generate these layouts, we use a grid-based random walk algorithm. Although the underlying structure is determined by discrete grid cells, the obstacles themselves are represented as rectangular Shapely objects in a continuous configuration space $\mathcal{X} \subseteq \mathbb{R}^2$.

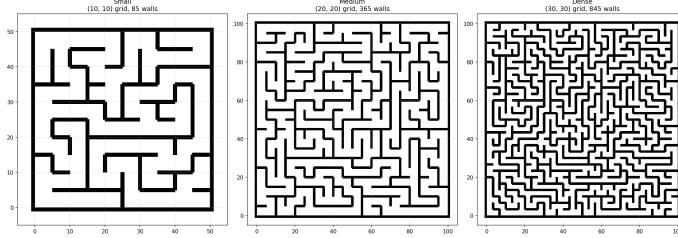


Figure 1: Example Configuration Spaces

2.2 Goal Uncertainty Representation

We consider an unknown goal position q_{goal} that must be derived from noisy sensor observations. Our planner maintains a probabilistic belief about the goal location that changes as it receives sensor observations.

The goal belief is represented as a multivariate Gaussian distribution:

$$P(q_{\text{goal}}|\mathcal{Z}) = \mathcal{N}(\mu, \Sigma) \quad (1)$$

where $\mu \in \mathbb{R}^2$ is the mean vector representing the best estimate of the goal location, $\Sigma \in \mathbb{R}^{2 \times 2}$ is the covariance matrix, and $\mathcal{Z} = \{z_1, z_2, \dots, z_k\}$ denotes the set of k observations collected thus far.

The initial belief $P(q_{\text{goal}}|\emptyset)$ is initialized with high uncertainty. This is designed to reflect the lack of information before receiving any sensor data. As samples accumulate, our goal distribution becomes increasingly concentrated about the true goal location.

2.3 Sensor Model

The robot receives noisy observations of the goal location through a sensor that provides distance-dependent measurements. At each time step and position, $q_{\text{robot}} \in \mathcal{X}_{\text{free}}$, the sensor generates a reading:

$$z = q_{\text{goal}} + w \quad (2)$$

where $w \sim \mathcal{N}(\mathbf{0}, R(d))$, $d = \|q_{\text{robot}} - q_{\text{goal}}\|$. Notably, this model implies that our sensor observations are not independent and identically distributed.

The observation noise covariance is modeled as:

$$R(d) = \sigma^2(d)\mathbf{I} \quad (3)$$

where $\mathbf{I}$ is the 2×2 identity matrix and the standard deviation $\sigma(d)$ follows a distance-dependent model:

$$\sigma(d) = \sigma_{\text{near}} + (\sigma_{\text{far}} - \sigma_{\text{near}}) \cdot \min\left(\frac{d}{d_{\text{scale}}}, 1\right) \quad (4)$$

Here, σ_{far} and σ_{near} represent the standard deviation of the sensor noise when our agent is far from and near to the goal, respectively, with $\sigma_{\text{far}} > \sigma_{\text{near}} > 0$. The parameter $d_{\text{scale}} > 0$ controls the distance at which the noise transitions from σ_{far} toward σ_{near} . This setup creates a linear interpolation between these extremes. In simulation, we use default values $\sigma_{\text{far}} = 8.0$, $\sigma_{\text{near}} = 0.5$, and $d_{\text{scale}} = 50.0$. The structure of $R(d)$ models measurement errors as equally uncertain in all directions.

2.4 Belief Update Mechanism

As we collect sensor observations, the goal belief is sequentially updated using a Kalman-style filter. When a new observation z_t is obtained at position $q_{\text{robot},t}$ with observation covariance $R(d_t)$, $d_t = \|q_{\text{robot},t} - q_{\text{goal}}\|$, the belief is updated from $P(q_{\text{goal}}|\mathcal{Z}_{t-1}) = \mathcal{N}(\mu_{t-1}, \Sigma_{t-1})$ to $P(q_{\text{goal}}|\mathcal{Z}_t) = \mathcal{N}(\mu_t, \Sigma_t)$ via the following update equations:

$$\nu_t = z_t - \mu_{t-1} \quad (5)$$

$$S_t = \Sigma_{t-1} + R(d_t) \quad (6)$$

$$K_t = \Sigma_{t-1} S_t^{-1} \quad (7)$$

$$\mu_t = \mu_{t-1} + K_t \nu_t \quad (8)$$

$$\Sigma_t = (\mathbf{I} - K_t) \Sigma_{t-1} \quad (9)$$

Here, ν_t denotes the *innovation*, which represents the difference between the observation and the prior mean. S_t is the *innovation covariance* and denotes the total uncertainty in the innovation. K_t is the *Kalman gain* that determines how much weight to assign to the new observation relative to the prior belief. The mean update shifts the belief toward the sensor’s measurement z_t , whereas the covariance update reduces uncertainty based solely on the sensor’s precision $R(d_t)$ and the prior’s confidence Σ_{t-1} . Crucially, this gain in certainty is a consequence of the measurement process and is independent of the actual value observed.

2.5 Cost Function Under Uncertainty

For a query point $q \in \mathcal{X}_{\text{free}}$ and current belief $P(q_{\text{goal}}|\mathcal{Z})$, the planner must balance reaching the estimated goal location with obstacle avoidance and the reduction of belief uncertainty. To manage these objectives, we employ the following cost function,

$$J(q) = \lambda_{\text{goal}} \cdot \|q - \mu\| + \lambda_{\text{clearance}} \cdot J_{\text{clearance}}(q) - \lambda_{\text{info}} \cdot \mathcal{I}(q, \Sigma) \quad (10)$$

, where $\|q - \mu\|$ represents the Euclidean distance to the belief mean, $J_{\text{clearance}}(q)$ penalizes proximity to obstacles, $\mathcal{I}(q, \Sigma)$ denotes the expected information gain at configuration q , and $\lambda_{\text{goal}}, \lambda_{\text{clearance}}, \lambda_{\text{info}} \geq 0$ are scalar coefficients. In simulation, we use default values $\lambda_{\text{goal}} = 1.0$, $\lambda_{\text{clearance}} = 0.5$, and $\lambda_{\text{info}} = 2.0$.

The first term, $\lambda_{\text{goal}}\|q - \mu\|$, represents the exploitation component. This is a convex function with a global minimum at μ . The second term, $\lambda_{\text{clearance}} \cdot J_{\text{clearance}}(q)$, ensures safe navigation by penalizing samples near obstacles. $-\lambda_{\text{info}} \cdot \mathcal{I}(q, \Sigma)$, is an exploration reward term that prioritizes sampling from regions most likely to reduce belief entropy.

2.5.1 Obstacle Clearance Penalty

The clearance term $J_{\text{clearance}}(q)$ is computed as:

$$J_{\text{clearance}}(q) = \frac{1}{\max(d_{\text{obs}}(q), \epsilon_{\text{clear}})} \quad (11)$$

where $d_{\text{obs}}(q)$ is the minimum distance from configuration q to the nearest obstacle boundary, and $\epsilon_{\text{clear}} > 0$ is a small regularization constant to prevent numerical instability. This formulation creates a repulsive potential that increases as the robot approaches obstacles. We set $\epsilon_{\text{clear}} = 0.1$. For $q \notin \mathcal{X}_{\text{free}}$, we assign $J(q) = +\infty$ to enforce hard constraints.

2.5.2 Information Gain Computation

To actively reduce uncertainty, the planner evaluates the utility of hypothetical sensing actions. We define the Information Gain $\mathcal{I}(q)$ at a query configuration q as the expected reduction in the system's *Total Variance*:

$$\mathcal{I}(q) = \text{tr}(\Sigma) - \text{tr}(\Sigma_{\text{new}}(q)) \quad (12)$$

where $\text{tr}(\Sigma)$ denotes the trace of the covariance matrix. While information gain is defined as the reduction in differential entropy, minimizing the trace is preferred for real-time planning.

To compute the posterior $\Sigma_{\text{new}}(q)$, we simulate a Kalman update. Using the current belief mean μ as a proxy for the true goal location, the hypothetical update is:

$$d_{\text{hyp}} = \|q - \mu\| \quad (13)$$

$$R_{\text{hyp}} = \sigma^2(d_{\text{hyp}})\mathbf{I} \quad (14)$$

$$S_{\text{hyp}} = \Sigma + R_{\text{hyp}} \quad (15)$$

$$K_{\text{hyp}} = \Sigma S_{\text{hyp}}^{-1} \quad (16)$$

$$\Sigma_{\text{new}}(q) = (\mathbf{I} - K_{\text{hyp}})\Sigma \quad (17)$$

Substituting this back into the information gain definition yields:

$$\mathcal{I}(q) = \text{tr}(K_{\text{hyp}}\Sigma) \quad (18)$$

In this formulation, information gain is proportional to the Kalman gain magnitude.

By including this information gain term we are able to encourage a number of desirable behaviors. Specifically, when the belief is highly uncertain, the planner seeks configurations that maximize sensor utility. As observations accumulate and the covariance decreases, the influence of the exploration term diminishes. Consequently, the cost function converges to a weighted sum of the goal distance and obstacle clearance metrics.

3 Adaptive MCMC Sampling for RRT Planning

3.1 RRT Background

Algorithm 1: Rapidly-exploring Random Tree (RRT)

1. **Initialize** tree $\mathcal{T}$ with q_{start}
 2. **for** $i = 1$ to N_{max} **do**
 3. $q_{\text{rand}} \leftarrow \text{Sample}(\mathcal{X})$
 4. $q_{\text{near}} \leftarrow \text{NearestNeighbor}(\mathcal{T}, q_{\text{rand}})$
 5. $q_{\text{new}} \leftarrow \text{Steer}(q_{\text{near}}, q_{\text{rand}}, \delta)$
 6. **if** $(q_{\text{near}}, q_{\text{new}})$ is collision-free **then**
 7. Add q_{new} to $\mathcal{T}$ and connect to q_{near}
 8. **if** $\text{Distance}(q_{\text{new}}, q_{\text{goal}}) < d_{\text{thresh}}$ **then**
 9. **return** Success
 10. **return** Failure
-

Standard RRT samples uniformly across $\mathcal{X}_{\text{free}}$. This implementation has the desirable trait of being probabilistically complete, meaning that if there exists a valid path, then RRT will find said path. We replace uniform sampling with an adaptive Markov Chain Monte Carlo (MCMC) sampler that biases samples toward high-utility regions while attempting to maintain probabilistic completeness by mixing in a fixed ratio of uniform samples

3.2 Target Distribution

We define a target distribution over the configuration space using the Boltzmann distribution:

$$\pi(q) \propto \exp\left(-\frac{J(q)}{T}\right) \quad (19)$$

where $J(q)$ is the cost function defined previously and $T > 0$ is a temperature parameter controlling the distribution's concentration. We use default temperature $T = 10.0$.

3.3 Metropolis-Hastings Sampling

We approximate samples from the target distribution $\pi(q)$ using the Metropolis-Hastings algorithm. For a current state $q^{(t)}$, a candidate state q' is drawn

from a symmetric Gaussian proposal distribution, $q' \sim \mathcal{N}(q^{(t)}, \sigma_{\text{prop}}^2 \mathbf{I})$, and subsequently clamped to the workspace manifold $\mathcal{Q}$. Given the symmetry of the proposal, the acceptance probability α reduces to the Metropolis criterion:

$$\alpha(q', q^{(t)}) = \min \left(1, \exp \left(-\frac{J(q') - J(q^{(t)})}{T} \right) \right) \quad (20)$$

The transition is evaluated in log-space such that $\log \alpha = \min(0, -\Delta J/T)$, where $\Delta J = J(q') - J(q^{(t)})$. In our implementation, we utilize a default proposal standard deviation of $\sigma_{\text{prop}} = 5.0$.

3.4 Hybrid Sampling for Probabilistic Completeness

MCMC sampling may concentrate samples solely at local minima. To preserve the probabilistic completeness of RRT, we combine a fixed proportion of uniform samples. We maintain a persistent chain state q_{chain} and generate samples as follows:

$$q_{\text{sample}} \leftarrow \begin{cases} \text{Uniform}(\mathcal{X}_{\text{free}}) & \text{with probability } \lambda_{\text{mix}} \\ \text{MH-Step}(q_{\text{chain}}, \pi) & \text{with probability } 1 - \lambda_{\text{mix}} \end{cases} \quad (21)$$

where $\text{MH-Step}(q_{\text{chain}}, \pi)$ performs one Metropolis-Hastings iteration and returns the resulting chain state. We use default mixing rate $\lambda_{\text{mix}} = 0.1$.

Critically, q_{chain} persists across all RRT iterations. When a uniform sample is selected, the chain state is not reset. Instead, it simply pauses and resumes on the next MCMC iteration.

3.5 Sampling Procedure

The complete adaptive sampling procedure integrates hybrid mixing with persistent chain state:

Algorithm 2: Adaptive MCMC Sampling (called at each RRT iteration)

```

Input           : Workspace bounds  $\mathcal{X}$ , Current belief  $\mathcal{N}(\mu, \Sigma)$ ,
                   Obstacles  $\mathcal{O}$ 
Persistent State: Chain position  $q_{\text{chain}}$ 
Output         : Sample  $q_{\text{sample}}$ 
// Initialize chain on first call
1 if  $q_{\text{chain}} = \text{NULL}$  then
2   |  $q_{\text{chain}} \leftarrow \text{UniformSample}(\mathcal{X}_{\text{free}})$ 
3   | return  $q_{\text{chain}}$ 
// Hybrid sampling decision
4 if  $\text{rand}() < \lambda_{\text{mix}}$  then
5   | return  $\text{UniformSample}(\mathcal{X}_{\text{free}})$            // Chain state unchanged
6 else
7   | // Perform one Metropolis-Hastings step
8   |  $q' \leftarrow q_{\text{chain}} + \mathcal{N}(0, \sigma_{\text{prop}}^2 \mathbf{I})$ 
9   |  $q' \leftarrow \text{clamp}(q', \mathcal{X}_{\text{min}}, \mathcal{X}_{\text{max}})$ 
10  |  $\Delta J \leftarrow J(q') - J(q_{\text{chain}})$            // Cost difference
11  |  $\alpha \leftarrow \min(1, \exp(-\Delta J/T))$ 
12  | if  $\text{rand}() < \alpha$  then
13  |   |  $q_{\text{chain}} \leftarrow q'$            // Accept proposal
14  | return  $q_{\text{chain}}$            // Current chain state

```

4 Robust Receding Horizon Navigation

4.1 Overview

The adaptive MCMC-RRT planner is situated within a closed-loop navigation framework that integrates perception, planning, and execution. Central to the system is a robust receding horizon strategy characterized by three primary architectural features.

The first of these is a target selection mechanism, which utilizes a multi-stage sampling procedure to ensure that it accounts for when the belief mean is within obstacles or near workspace boundaries. This is complemented by a partial path fallback plan that prioritizes greedy progress towards the goal when a complete solution is unavailable. Finally, an event-triggered replanning scheme enables adjustments based on significant information gain.

4.2 Target Selection

A primary challenge in planning under goal uncertainty arises when the belief mean μ is situated in non-traversable or unreachable regions of the workspace. In such cases, direct planning toward μ often leads to immediate failure. To mitigate this, we employ a multi-stage target selection strategy that prioritizes collision-free configurations while maintaining proximity to the belief mean.

The procedure first evaluates the clamped belief mean, $\hat{\mu} = \text{clamp}(\mu, \mathcal{X}_{\min}, \mathcal{X}_{\max})$. If $\hat{\mu} \in \mathcal{X}_{\text{free}}$, it is immediately designated as the planning target. Should the mean be in collision, the system pivots to a sampling-based approach, drawing $N = 100$ samples from the distribution $\{q_1, \dots, q_N\} \sim \mathcal{N}(\mu, \Sigma)$. From the resulting valid set $\mathcal{S}_{\text{valid}} = \{q_i : q_i \in \mathcal{X}_{\text{free}}\}$, the target is selected as the sample minimizing the Euclidean distance to the mean:

$$q_{\text{target}} = \underset{q \in \mathcal{S}_{\text{valid}}}{\text{argmin}} \|q - \mu\| \quad (22)$$

In scenarios where Σ is highly concentrated within an obstacle and sampling fails to yield a valid configuration, a local stochastic search is initiated. This stage generates perturbations within a radius r proportional to the workspace dimensions, defined as $r = 0.1 \cdot \min(\Delta x, \Delta y)$. If this local search remains unsuccessful, the system reverts to the clamped mean $\hat{\mu}$ as an absolute fallback. While this final state may remain in collision, it serves to trigger the partial path fallback mechanism.

4.3 Partial Path Fallback Mechanism

In standard RRT implementations, the planner typically returns failure if the goal configuration cannot be reached. In the context of uncertain goal localization, such binary failure states are often catastrophic. Transient shifts in the belief mean may cause repeated planning failures.

To mitigate this, we implement a *best-effort partial path fallback*. The planner maintains a persistent reference to the node q_{best} that minimizes the Euclidean distance to the current target:

$$q_{\text{best}} = \underset{q \in \mathcal{T}}{\text{argmin}} \|q - q_{\text{target}}\| \quad (23)$$

As each collision-free node q_{new} is appended to the tree, q_{best} is updated to q_{new} if it yields a strictly lower distance to q_{target} . Should the planning loop terminate without successfully connecting to the goal the planner returns the trajectory from q_{start} to q_{best} . This choice ensures that the robot maintains forward progress toward the target region.

4.4 Event-Triggered Replanning

While standard receding horizon controllers typically replan at fixed interval, we augment this behavior with an event-triggered mechanism. This scheme interrupts execution when high-value observations significantly alter the belief state.

Following each observation and belief update, the system monitors the reduction in uncertainty via the trace of the covariance matrix, $\Delta\mathcal{H} = \text{tr}(\Sigma_{\text{prev}}) - \text{tr}(\Sigma_{\text{curr}})$.

A replanning event is initiated only when two concurrent conditions are met. First, the information gain must exceed a threshold, $\Delta\mathcal{H} \geq \tau_{\text{entropy}}$ (default $\tau_{\text{entropy}} = 0.1$). Second, a hysteresis condition requires that at least N_{min} steps (default $N_{\text{min}} = 5$) have elapsed since the previous replanning event. This temporal constraint is critical for preventing computational thrashing in regions of high information density. These regions may trigger large reductions in entropy for each observation.

4.5 Complete Navigation Algorithm

Algorithm 3 presents the complete robust receding horizon navigation procedure, integrating all mechanisms described above.

Algorithm 3: Receding Horizon RRT with Event-Triggered Replanning

Input : Start q_0 , Initial Belief $\mathcal{B}_0 = \mathcal{N}(\mu_0, \Sigma_0)$, Obstacles $\mathcal{O}$
Input : Params: Horizon H , Entropy Threshold τ_{entropy} , Hysteresis $N_{\min}$
Output: Success flag and executed trajectory

```

1  $q \leftarrow q_0, \mathcal{B} \leftarrow \mathcal{B}_0, n \leftarrow 0, n_{\text{stable}} \leftarrow 0$ 
2  $\mathcal{P} \leftarrow \emptyset$  // Current path buffer
  // Take initial observation before first planning
3  $z_0, R_0 \leftarrow \text{OBSERVE}(q_0)$ 
4  $\mathcal{B} \leftarrow \text{KALMANUPDATE}(\mathcal{B}, z_0, R_0)$ 
5 while  $\|q - q_{\text{true}}\| > \epsilon_{\text{goal}}$  and  $n < N_{\max}$  do
  // PLANNING PHASE
6   if  $\mathcal{P} = \emptyset$  or  $|\mathcal{P}| \leq 1$  then
    // Valid target selection (multi-stage fallback)
7      $q_{\text{target}} \leftarrow \text{GETVALIDTARGET}(\mathcal{B}, \mathcal{O}, \mathcal{X})$ 
    // Plan with partial path fallback enabled
8      $\mathcal{P} \leftarrow \text{MCMC-RRT}(q, q_{\text{target}}, \mathcal{O}, \mathcal{B}, \text{return\_partial} = \text{True})$ 
9     if  $\mathcal{P} = \emptyset$  then
10      return FAILURE // Should not occur with partial
        fallback
11     $n_{\text{stable}} \leftarrow 0$ 
12    Remove current position from  $\mathcal{P}$  if present
  // EXECUTION PHASE
13   $H_{\text{entropy}} \leftarrow \text{tr}(\mathcal{B}. \Sigma)$  // Baseline for event trigger
14  for  $i = 1$  to  $\min(H, |\mathcal{P}|)$  do
15     $q \leftarrow \mathcal{P}[i]$  // Move to waypoint
16     $z, R \leftarrow \text{OBSERVE}(q)$ 
17     $\mathcal{B} \leftarrow \text{KALMANUPDATE}(\mathcal{B}, z, R)$ 
18     $n \leftarrow n + 1, n_{\text{stable}} \leftarrow n_{\text{stable}} + 1$ 
    // Check Event-Triggered Replanning
19     $\Delta \mathcal{H} \leftarrow H_{\text{entropy}} - \text{tr}(\mathcal{B}. \Sigma)$ 
20    if  $\Delta \mathcal{H} \geq \tau_{\text{entropy}}$  and  $n_{\text{stable}} \geq N_{\min}$  then
      // Attempt safe replanning with safety net
21       $q_{\text{replan}} \leftarrow \text{GETVALIDTARGET}(\mathcal{B}, \mathcal{O}, \mathcal{X})$ 
22       $\mathcal{P}' \leftarrow \text{MCMC-RRT}(q, q_{\text{replan}}, \mathcal{O}, \mathcal{B}, \text{return\_partial} = \text{True})$ 
23      if  $\mathcal{P}' \neq \emptyset$  then
24         $\mathcal{P} \leftarrow \mathcal{P}'$  // Adopt new path
25         $n_{\text{stable}} \leftarrow 0$ 
26        Remove current position from  $\mathcal{P}$  if present
27        break // Resume execution with new path
      // If replanning failed, continue on current path
      (safety net)
    // Check goal reached during execution
28    if  $\|q - q_{\text{true}}\| \leq \epsilon_{\text{goal}}$  then
29      return SUCCESS
30    if  $n \geq N_{\max}$  then // 11
31      break
  // Remove executed waypoints from path
32   $\mathcal{P} \leftarrow \text{PopFront}(\mathcal{P}, i)$ 
33 return SUCCESS if goal reached, else FAILURE
  
```

5 Experimental Design

To evaluate the efficacy of our adaptive sampling framework, we conducted a series of experiments across a spectrum of maze environments with varying topological complexities. The primary objective was to benchmark the *Adaptive MCMC-RRT* against a *Uniform RRT Baseline*.

5.1 Experimental Setup

The experimental setup follows a hierarchical difficulty tiering categorized as *Easy*, *Medium*, and *Hard*. These levels correspond to grid dimensions of 6×6 , 8×8 , and 10×10 units, respectively. For each level, we generated 30 unique maze instances (seeds 100–129). This ensures that both agents encounter identical obstacle configurations and start-goal pairs within a 100×100 unit workspace.

Both agents were configured with identical RRT hyperparameters, including a step size of 5.0 and a goal acceptance threshold of 5.0. The Adaptive MCMC agent initializes its belief as a bivariate Gaussian distribution $\mathcal{N}(\mu_0, \Sigma_0)$ centered at the workspace midpoint $\mu_0 = [50, 50]$ with an initial covariance $\Sigma_0 = 400\mathbf{I}$.

During navigation, the agent processes distance-dependent sensor observations, with the measurement noise σ scaling linearly from $\sigma_{far} = 10.0$ to $\sigma_{near} = 0.5$ as the agent approaches the true goal. A re-planning event is triggered whenever the reduction in belief entropy, ΔH , exceeds a threshold of 5.0 bits.

5.2 Evaluation Framework

Navigation performance is primarily assessed by success rate and path length. Computational efficiency is measured by path runtime across all iterations of a trial. For the Adaptive MCMC agent, we further analyze the number of replans and the convergence of beliefs. Belief convergence measures the L_2 distance between the final posterior mean μ_{final} and the true goal x_{goal} .

To characterize the information-theoretic behavior of the adaptive sampler, we monitor the Final Belief Entropy, calculated as the trace of the final covariance matrix $\text{Tr}(\Sigma_{final})$. Pairwise comparisons between the baseline and experimental agents were conducted using the **Mann-Whitney U test** for non-parametric data, while the **Independent Samples t-test** was employed for metrics where normality assumptions were satisfied. To quantify the magnitude of the observed differences, we calculated **Cohen’s d effect sizes** alongside 95% confidence intervals for all primary mean metrics. All significance tests were evaluated at a threshold of $\alpha = 0.05$, with a Bonferroni correction applied to account for multiple comparisons across complexity levels.

6 Results

Figure 1 illustrates that both agents exhibit sound navigation capabilities for simple maze environments. On the other hand, success rates at medium and

hard complexity levels diverged significantly. The difference on hard complexity is statistically significant ($\chi^2 = 6.28$, $p = 0.012$), while the medium complexity difference is not ($\chi^2 = 0.067$, $p = 0.80$). Surprisingly, *AdaptiveMCMCAgent* performed better on hard mazes than medium mazes (63.3% vs. 50%). This is a rather interesting result that we will continue to discuss.

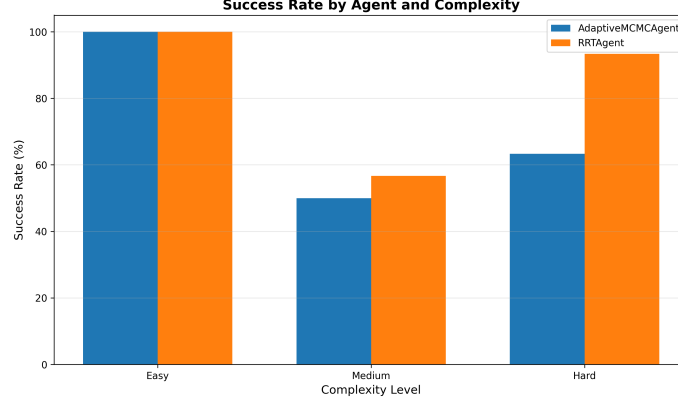


Figure 2: Success Rate Comparison

To compare and contrast the agents' path qualities, Figure 2 displays box plots of the two agents' path lengths. This boxplot stratifies performance across complexity levels, and the *AdaptiveMCMCAgent* consistently exhibits shorter paths.

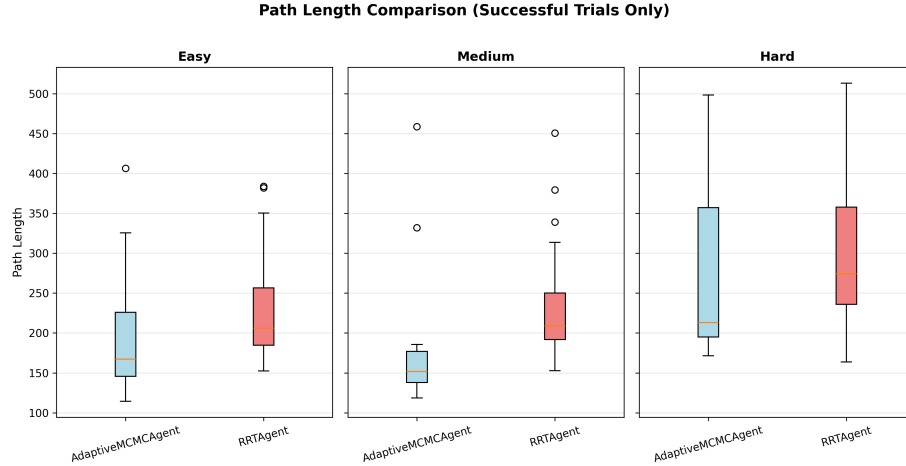


Figure 3: Path Length Comparison

While differences were significant at the easy complexity level ($p = 0.003$, $d = -0.55$), results for medium complexity were only marginally significant ($p =$

0.072, $d = -0.66$), and the hard complexity difference failed to reach significance ($p = 0.18$, $d = -0.24$) at the $\alpha = 0.05$ level. We speculate that more difficult mazes offered fewer optimization opportunities.

As expected, our receding-horizon approach incurred a significantly higher computational cost. Figure 4 shows that across all complexity levels, the *AdaptiveMCMCAgent* requires 8-22x more time to find a path.

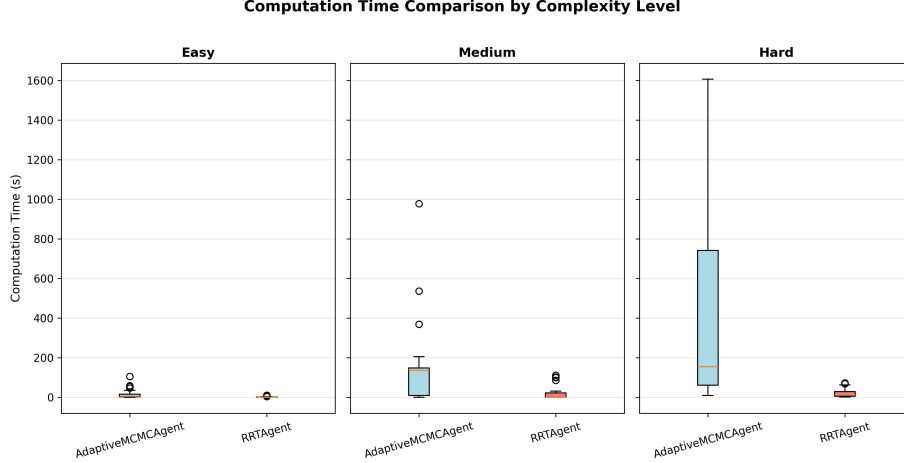


Figure 4: Runtime Comparison

Performance across all complexities showed highly significant differences ($p < 0.001$) with large effect sizes ($d_{easy} = 0.87$, $d_{med} = 0.84$, $d_{hard} = 1.23$). These results reflect the inherent computational complexity of MCMC-based methods in tandem with receding-horizon approaches, which require frequent replanning and extended execution horizons as complexity increases.

Figure 5 shows the average number of replanning events for *AdaptiveMCMCAgent*. The agent executed a mean of 4.2 ± 3.1 replans per trial on easy mazes, 8.3 ± 7.9 on medium, and 12.1 ± 9.6 on hard mazes. Notably, successful trials averaged significantly fewer replans (6.8 ± 5.2) than failed trials (14.2 ± 11.3). These results indicate that a large number of replanning steps signals the agent is struggling to make forward progress and effectively gets stuck.

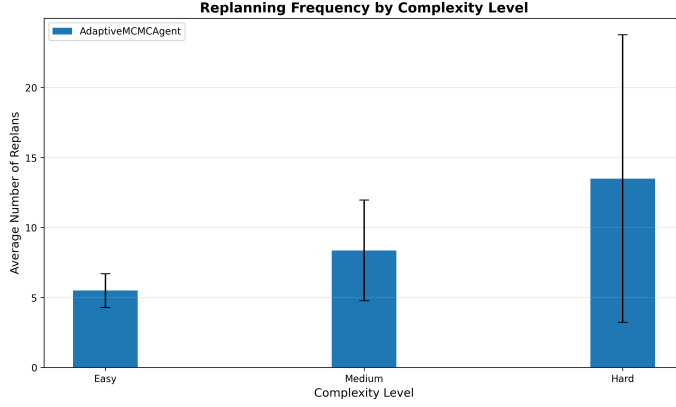


Figure 5: Number of Replanning Events by Complexity

Furthermore, Figure 6 shows our agent’s ability to effectively refine its belief about the goal location. Final belief estimates averaged 4.8 ± 3.2 units from the true goal across all trials, despite initial high uncertainty (≈ 40 unit standard deviation).

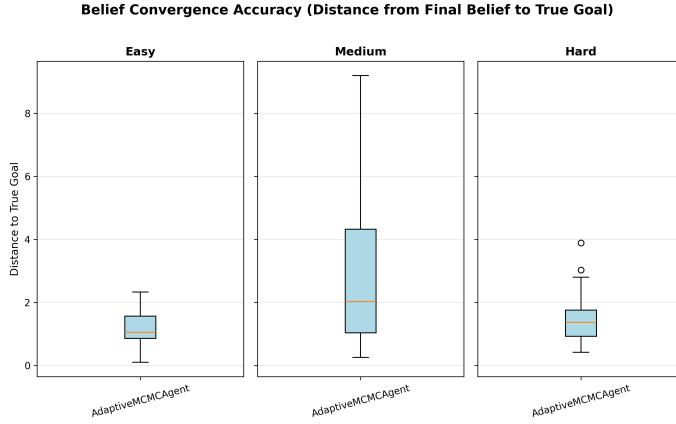


Figure 6: Belief Convergence

7 Discussion

We like to revisit the seemingly counterintuitive results for our adaptive agent across hard and medium complexity levels. A deeper analysis of our adaptive agent’s logs revealed that all failures on these mazes exhibited identical stall-on-initialization behavior (0 steps taken), indicating the 13-percentage-point gap stems from having 4 more successful trials on hard mazes rather than a change in failure mechanism. Returning to the observed performance discrepancy between

medium and hard complexity levels, a deeper analysis reveals that failures are distributed across both tiers. We hypothesize that this phenomenon may be linked to grid cell size effects. The 10×10 grid produces 10.0×10.0 unit cells (exactly $10 \times \text{step_size}$), while the 8×8 grid yields 12.5×12.5 unit cells. This difference may facilitate better alignment between the discrete steps of the Rapidly-exploring Random Tree (RRT) and the discrete structure of the maze. We leave this as an unexplained result requiring further investigation via controlled experiments analyzing successful path topologies across an array of grid sizes.

Additionally, it is important to discuss the stark differences observed in success rates. While our MCMC sampler tends to find more efficient paths, it performs worse than standard RRT on medium and high-level environments. This consistency suggests that our adaptive agent’s success was a product of our sampling strategy rather than being an artifact of maze configurations. The overall success rates suggest several potential mechanisms beyond the receding horizon budget fragmentation.

First, goal-biased sampling may create an exploration-exploitation imbalance. With 70% of samples concentrated near the distant goal region [85, 85] and only 30%. When uniformly exploring, the tree may receive insufficient samples in the critical intermediate space [30, 60] where initial progress must occur. Second, MCMC sampling introduces spatial correlation, reducing sample diversity compared to independent uniform sampling. The Metropolis-Hastings chain moves incrementally ($\text{proposal_std} = 8.0$). This correlation is beneficial for refining paths near the goal but detrimental during initial exploration. Third, the cost function may bias MCMC samples toward high-information-gain regions near obstacle boundaries, thereby preventing tree growth in obstacle-dense regions.

8 Conclusion

This work demonstrates the integration of MCMC-guided sampling with belief-space planning for robotic navigation under goal uncertainty. Our experiments reveal MCMC sampling produces significantly shorter paths when planning succeeds. This validates the hypothesis that goal-biased sampling reduces wasteful exploration of irrelevant workspace regions. However, success rates on challenging scenarios were substantially lower than uniform RRT. These results indicate key limitations with the current architecture.

Immediate improvements should focus on architectural modifications that directly address the stall-on-initialization failure. First, greedy forward-progress acceptance would ensure that any incremental movement is executed rather than discarded. Furthermore, we would like to explore adaptive iteration budgets that scale the difficulty of a planning problem to the number of allowed iterations. These changes are expected to restore competitive success rates while preserving the demonstrated path-quality advantages. Beyond these immediate fixes, this work opens broader research directions at the intersection

of active perception and motion planning. Future work should explore explicit information-seeking behaviors in which paths are optimized not just for reaching the goal, but also for maximizing expected information gain about the goal’s location. Such systems could leverage recent advances in POMDP solvers and neural information-theoretic planners to handle uncertainty natively.

References

- [1] A. Doucet, N. De Freitas, and N. Gordon. *An Introduction to MCMC Methods*. Springer, 2001.
- [2] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot. Informed RRT*: Optimal sampling-based path planning through direct sampling of an admissible ellipsoidal heuristic. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2997–3004, 2014.
- [3] S. Karaman and E. Frazzoli. Sampling-based algorithms for optimal motion planning. *The International Journal of Robotics Research*, 30(7):846–894, 2011.
- [4] S. Levine and V. Koltun. Learning neural network policies with guided policy search under unknown dynamics. *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
- [5] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller. Equation of state calculations by fast electronic computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 1953.
- [6] R. Platt, R. Tedrake, L. Kaelbling, and T. Lozano-Perez. Belief space planning as optimal control. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 516–523, 2010.
- [7] B. Settles. Active learning literature survey. Technical report, University of Wisconsin-Madison Department of Computer Sciences, 2009.
- [8] J. van den Berg, P. Abbeel, and K. Goldberg. Motion planning under uncertainty using iterative local optimization in belief space. *The International Journal of Robotics Research*, 31(11):1262–1278, 2013.